

Caso real Netflix — questões para discussão

Disciplina: Arquitetura de Software — Módulo 3: Serviços

Caso: Netflix: sete anos para reconstruir, dois segundos para começar o filme

Fonte do caso: <https://marco-mendes.github.io/arquitetura-software/modulo-3-servicos/casos-reais/>

Autor: pedrojovanuzzi — 01/09/2026

1. Segundo o anúncio oficial, o que aconteceu em 2008 e qual operação da empresa ficou parada por três dias?

Em agosto de 2008 a Netflix sofreu uma corrupção grave do seu banco de dados relacional central — um banco único, escalado verticalmente, hospedado no datacenter da própria empresa. O anúncio oficial de fevereiro de 2016 resume o episódio em uma frase: *"We experienced a major database corruption and for three days could not ship DVDs to our members."*

A operação interrompida foi a **expedição de DVDs pelo correio**, que na época era o negócio principal da empresa. Durante três dias nenhum disco foi enviado a nenhum assinante.

O detalhe arquitetural que importa não é a duração, e sim o **formato** da falha: não houve degradação parcial, região afetada nem subconjunto de clientes atingido. O sistema inteiro parou, porque havia um único componente cuja perda derrubava tudo — o retrato clássico de um ponto único de falha. Dessa experiência a empresa tirou duas conclusões, e a segunda costuma ser esquecida:

- 1 escalar verticalmente um banco único era uma aposta perdida;
- 2 comprar e instalar servidores num ritmo compatível com o crescimento do streaming era um problema que a Netflix não queria resolver.

A primeira conclusão levou à reescrita da arquitetura; a segunda, à decisão de sair do datacenter próprio. São decisões independentes, e a migração só faz sentido quando se lê as duas juntas.

2. A Netflix declara ter reconstruído a tecnologia em vez de transportá-la. Explique a diferença entre as duas coisas e o que cada uma muda na arquitetura resultante.

Transportar (o que o mercado chama de *lift-and-shift*) é mover as máquinas virtuais e os processos existentes para a infraestrutura de um provedor, preservando a arquitetura. Troca-se o dono do datacenter e o modelo de custo — deixa-se de comprar servidores e passa-se a alugar capacidade sob demanda —, mas a topologia viaja intacta: o banco central continua central, o monólito continua monólito, os acoplamentos continuam onde estavam e o ponto único de falha de 2008 é reproduzido fielmente na nuvem. Ganha-se elasticidade de provisionamento; não se ganha disponibilidade.

Reconstruir é o que a Netflix declara ter feito ao adotar *"a cloud-native approach, rebuilding virtually all of our technology"*: trocar o banco relacional central por armazenamentos NoSQL sob propriedade de cada serviço, quebrar o monólito em serviços independentes e assumir como premissa de projeto que instâncias, zonas e regiões inteiras falham.

O que cada uma muda na arquitetura resultante:

Aspecto	Transportar	Reconstruir
Propriedade dos dados	banco compartilhado; a fronteira depende de disciplina	um armazenamento por serviço; a fronteira é estrutural
Modo de falha típico	falha global: um componente derruba o conjunto	falha parcial: o sistema degrada em partes
Unidade de mudança	o sistema inteiro	um serviço por vez, com ciclo próprio
Prazo e custo	curto e barato	longo e caro: sete anos, de 2008 a janeiro de 2016

Foi a reescrita — e não a mudança de provedor — que produziu os resultados que a empresa declara: oito vezes mais assinantes de streaming do que em 2008, visualização crescendo três ordens de grandeza em oito anos, disponibilidade se aproximando de quatro noves e custo de nuvem por início de reprodução equivalente a uma fração do custo em datacenter próprio. E foi a operação simultânea em múltiplas regiões da AWS, consequência da reconstrução, que permitiu ligar o serviço em mais de 130 países de uma só vez em 6 de janeiro de 2016.

A lição generalizável é que a nuvem entrega elasticidade e um modelo de custo diferente. Os atributos de qualidade que costumam justificar a migração — disponibilidade, escalabilidade horizontal, resiliência — vêm da arquitetura, não do provedor. Quem transporta leva junto os próprios defeitos e passa a pagar aluguel por eles.

3. O Open Connect inverte a lógica de cache: o conteúdo chega antes do pedido. Que propriedade do negócio da Netflix torna isso possível, e que tipo de serviço jamais conseguiria fazer o mesmo?

Um cache convencional é **reativo**: o primeiro usuário que pede um arquivo paga a conta de buscá-lo na origem e os seguintes aproveitam. O Open Connect é **proativo**: a documentação oficial descreve o preenchimento noturno dos aparelhos e a pré-carga do conteúdo apropriado para a região geográfica de destino, num processo que leva de uma a duas semanas na configuração inicial de cada equipamento. Quando o assinante aperta o play, o vídeo não está sendo buscado — ele já estava a poucos quilômetros dali.

A propriedade do negócio que torna isso possível tem três partes, e elas precisam valer ao mesmo tempo:

- **O catálogo é finito, conhecido de antemão e estável no curto prazo.** O conteúdo é pré-gravado e licenciado com antecedência. Existe uma lista fechada de arquivos, e ela não muda entre o preenchimento da madrugada e o consumo da noite seguinte.
- **A demanda é previsível e concentrada por região.** Uma fração pequena do catálogo responde por uma fração grande das horas assistidas, e o padrão regional se repete. Isso mantém a taxa de acerto alta mesmo com um aparelho de capacidade limitada — até 120 TB no modelo de armazenamento, até 60 TB no modelo global.
- **A demanda é, em parte, induzida pela própria empresa.** A recomendação influencia cerca de 80% das horas assistidas, então a Netflix não apenas prevê o que será assistido: ela participa da decisão.

Há ainda um facilitador de arquitetura, e não de negócio: a Netflix escreve o aplicativo para os mais de dois mil modelos de aparelho que suporta, o que permite empurrar para o cliente a medição de rede e a escolha do servidor entre os candidatos devolvidos pela nuvem.

Que tipo de serviço jamais conseguiria fazer o mesmo. Qualquer um em que o conteúdo não existe antes do pedido, ou existe em variedade grande demais para ser antecipado:

- **Transmissão ao vivo** — esporte, notícia, leilão: o próximo segundo de vídeo está sendo produzido agora; não há o que pré-carregar de madrugada. É por isso que o ao vivo é um problema de engenharia diferente do catálogo, mesmo dentro da própria Netflix.
- **Comunicação em tempo real** — videochamada, mensageria: o conteúdo é criado pelos próprios participantes no instante do uso.
- **Conteúdo gerado por usuário na cauda longa** — um vídeo publicado há dez minutos, ou um vídeo com cinco visualizações por mês, não justifica ocupar espaço em milhares de aparelhos; aqui o cache reativo é a única escolha economicamente defensável.
- **Respostas personalizadas ou transacionais** — busca, feed, saldo bancário, carrinho de compras, painel de indicadores: a resposta depende de quem pergunta e de quando, e é calculada por requisição.

O critério é sempre o mesmo: **cache proativo só compensa quando o conjunto de objetos é pequeno, conhecido antes do pedido e a demanda é concentrada o bastante para garantir taxa de acerto alta.** É uma decisão que se apoia numa propriedade do negócio, e não numa propriedade da tecnologia — e é por isso que ela não se copia junto com o software.

4. Compare Isthmus, arquitetura ativa-ativa e Chaos Kong quanto ao modo de falha que cada um endereça.

As três coisas costumam ser citadas juntas como "a resiliência da Netflix", mas ocupam papéis diferentes e obedecem a uma ordem de dependência.

Isthmus (2013) — um modo de falha específico e conhecido. Ataca a falha do balanceador de carga de uma região inteira: o tráfego de entrada é roteado por outra região enquanto o processamento continua na região original. É resiliência **parcial e cirúrgica** — resolve o componente que quebrou, não a perda da região. Se a região inteira desaparecesse, o Isthmus não teria para onde mandar o trabalho.

Arquitetura ativa-ativa (dezembro de 2013) — a perda de uma região inteira. Aqui a natureza da resposta muda: não é um remendo para um modo de falha, é uma **capacidade estrutural**. O serviço passa a rodar simultaneamente em mais de uma região da AWS, e cada região é capaz de atender os membros da outra. Isso obriga a resolver problemas que simplesmente não existem em uma região só — replicação do Cassandra entre regiões, invalidação de cache remoto no EVCache quando a escrita acontece do outro lado, e roteamento no Zuul capaz de mudar em tempo de execução. Em troca, cobre não apenas a falha do balanceador, mas qualquer falha regional, inclusive as que ninguém previu.

Chaos Kong — nenhum modo de falha; ele é o ensaio. O Chaos Kong evacua uma região inteira da AWS **com tráfego real rodando** e observa se a outra absorve. Não adiciona resiliência: **verifica empiricamente, em produção, se a resiliência que se acredita ter existe de fato.** Só se tornou possível depois da ativa-ativa, porque não se ensaia a perda de uma região quando só existe uma. E foi o ensaio que revelou o problema seguinte, invisível no papel: evacuar uma região levava cerca de

50 minutos, tempo demais para um incidente real. O Project Nimble reduziu esse tempo para 8 minutos — com uma equipe de duas pessoas em cerca de seis meses, número que interessa a quem for propor algo parecido.

Resumindo a comparação:

Critério	Isthmus	Ativa-ativa	Chaos Kong
Natureza	remendo dirigido	capacidade estrutural	exercício de verificação
Modo de falha coberto	balanceador de carga de uma região	perda de qualquer componente regional, inclusive a região inteira	nenhum: mede se os anteriores funcionam
Escopo da falha tolerada	parcial e conhecida	total e regional	—
Pré-requisito	uma segunda região para rotear a entrada	replicação de dados, invalidação de cache e roteamento entre regiões	ativa-ativa em operação real
O que revela	—	—	a diferença entre a resiliência projetada e a observada (50 min para 8 min)

A leitura arquitetural é que a resiliência não veio de uma ferramenta, e sim de uma progressão de cinco anos: primeiro trata-se um modo de falha conhecido, depois constrói-se a capacidade que generaliza a resposta, e só então ensaia-se a perda para descobrir o que a capacidade ainda não cobre. Pular etapas não funciona — e resumir tudo a "eles têm o Chaos Monkey" achata exatamente essa progressão.

5. O Chaos Monkey exige a plataforma Spinnaker e o Hystrix está em modo de manutenção desde 2018. Explique o que cada um desses fatos diz sobre reaproveitar a plataforma de ferramentas de outra empresa.

Chaos Monkey e o Spinnaker: a ferramenta pressupõe uma fundação que não vem junto. O próprio repositório declara a restrição que quase nunca aparece nos resumos: *"You must be managing your apps with Spinnaker to use Chaos Monkey to terminate instances."* A ferramenta assume a plataforma de entrega contínua da Netflix e, com ela, um conjunto de premissas implícitas: instâncias imutáveis e descartáveis, grupos de auto scaling capazes de repor o que foi encerrado, serviços sem estado local, observabilidade suficiente para perceber o efeito de uma instância morta e uma organização capaz de responder ao que a ferramenta encontrar. Encerrar máquinas em produção só é uma prática defensável quando essas condições valem; sem elas, o mesmo comando é apenas uma queda autoinfligida. Quem instala o Chaos Monkey sem o resto da fundação está copiando **o efeito visível de uma engenharia que não tem**. A justificativa oficial da ferramenta, aliás, deixa claro que o alvo é o comportamento das equipes: *"Exposing engineers to failures more frequently incentivizes them to build resilient services."*

Hystrix e o modo de manutenção: a dependência tem ciclo de vida próprio, alinhado ao interesse de quem a mantém. O Hystrix virou padrão de fato do disjuntor de circuito no ecossistema Java, e está em modo de manutenção desde novembro de 2018, com o próprio repositório indicando projetos ativos como o Resilience4j para novos desenvolvimentos. A empresa que abriu o código

seguir adiante quando as suas necessidades mudaram — e não deve manutenção a quem adotou a biblioteca. Adotar um componente de terceiro é assumir um risco de manutenção que não aparece na decisão inicial.

O que os dois fatos dizem juntos. São faces do mesmo erro: tratar a plataforma de ferramentas de outra empresa como um produto pronto, quando ela é o subproduto de um contexto específico.

- **Importe decisões e problemas, não artefatos.** O que se reaproveita de fato é o raciocínio: *por que* a Netflix precisou de um disjuntor, *que* problema o Chaos Monkey resolve, *que* condições tornam a prática segura. O padrão — disjuntor de circuito, isolamento de recursos, prazo de espera, degradação com resposta alternativa — sobrevive; a implementação é substituível, e o Resilience4j no lugar do Hystrix é a prova disso.
- **Toda ferramenta carrega pré-requisitos não declarados.** Antes de adotar, vale perguntar o que ela assume sobre entrega contínua, imutabilidade de instâncias, observabilidade e estrutura de equipes. O custo real quase nunca está na ferramenta; está na plataforma que a sustenta.
- **Um caso de referência é um retrato datado.** A plataforma de 2012 continua sendo apresentada em palestras como se fosse o estado atual da engenharia da Netflix, anos depois de partes dela terem sido aposentadas. Copiar o retrato é copiar respostas para perguntas que talvez a sua empresa nem tenha — e, na escala errada, o remédio custa mais que a doença.

Fontes utilizadas (citadas no próprio caso)

- Netflix, *Completing the Netflix Cloud Migration* (fevereiro de 2016) — incidente de 2008, os sete anos, a abordagem *cloud-native* e as métricas de resultado.
- Netflix, *Open Connect* e *Open Connect Appliances* — preenchimento noturno, pré-carga por região, mais de mil provedores parceiros e as especificações dos dois modelos de aparelho.
- Netflix Technology Blog, *Isthmus — Resiliency against ELB outages, Active-Active for Multi-Regional Resiliency* e *Project Nimble: Region Evacuation Reimagined*.
- Netflix, repositórios oficiais do *Chaos Monkey* e do *Hystrix* — origem das duas citações e do aviso de modo de manutenção.
- Carlos A. Gomez-Uribe e Neil Hunt, *The Netflix Recommender System: Algorithms, Business Value, and Innovation*, ACM Transactions on Management Information Systems, v. 6, n. 4, artigo 13, dezembro de 2015.